

รายงานโครงการพัฒนาโมเดลปัญญาประดิษฐ์ไทย

Thai Omni-Modal AI (OMNI)

การผสมผสานระบบเสียง ภาพ และข้อความ สู่ปัญญาประดิษฐ์ระดับแนวหน้าสำหรับภาษาไทย

จัดทำโดย: ทีมงานวิจัยและพัฒนา OMNI

ผู้ร่วมพัฒนา AI: Antigravity (Google DeepMind Team)

วันที่มีผล: 23 พฤษภาคม 2026

สถานะโครงการ: เฟส 1 (สำเร็จ) | เฟส 2 (กำลังดำเนินการ)

1. บทสรุปผู้บริหาร (Executive Summary)

โครงการ Thai Omni-Modal AI (OMNI) มีเป้าหมายในการพัฒนาแบบจำลองภาษาขนาดใหญ่ (LLM) ให้มีความสามารถระดับสากลในการรับรู้และตอบสนองหลายประสาทสัมผัส (Multi-Modal) สำหรับภาษาไทยโดยเฉพาะ โครงสร้างหลักประยุกต์ใช้โมเดลภาษา Qwen3.5-0.8B ประสานงานร่วมกับโมเดลถอดความเสียงระดับแนวหน้า Typhoon-Whisper-Turbo ของ SCB-10X โดยเชื่อมโยงผ่านโมดูลแปลงสัญญาณเสียง AudioProjector เพื่อเรียนรู้วิธีแปลงลักษณะของเสียงไทยเข้าไปยังระดับตัวแทนความรู้ (Hidden States) ของโมเดลภาษา

การพัฒนากำหนดขั้นตอนออกเป็น 4 เฟสหลัก เพื่อสร้างเสถียรภาพและคุณภาพสูงสุด ตั้งแต่การจัดตำแหน่งเสียง (Audio Alignment) การปรับแต่งคำสั่งแบบรวมประสาทสัมผัส (Omni SFT) การกลั่นกรององค์ความรู้ (Distillation) ไปจนถึงการส่งออกคำพูดเสียงไทยโดยตรง (Native Speech Output) รายงานฉบับนี้สรุปความคืบหน้าล่าสุดของการเอาชนะปัญหาข้อหาด้านข้อมูลเสียงและโครงสร้างทางสถาปัตยกรรมเพื่อเตรียมพร้อมสำหรับ SFT ระดับถัดไป

2. สถาปัตยกรรมระบบ (Architecture Blueprint)

สถาปัตยกรรมของ OMNI ถูกออกแบบให้มีความยืดหยุ่นสูง รองรับทั้งการทำงานแบบปกติ (Dense) และแบบก้ำวหน้า (Mixture-of-Experts) ดังรายละเอียดต่อไปนี้:

2.1 โครงสร้างปกติ (Dense Architecture)

รับสัญญาณเสียง (.wav) ผ่านโมเดล Typhoon-Whisper-Turbo แปลงลักษณะเสียงผ่าน MLP AudioProjector เพื่อผสานเข้ากับโทเค็นข้อความดิบ และส่งเข้าไปประมวลผลต่อใน Qwen3.5-0.8B เพื่อผลิตผลลัพธ์ข้อความภาษาไทย หรือส่งต่อไปยัง AudioLMHead (EnCodec) สำหรับผลิตไฟล์เสียงพูดตอบกลับ

2.2 โครงสร้างผู้เชี่ยวชาญแบบผสม (Sparse MoE Architecture)

เปลี่ยนชั้น FFN (Feed-Forward Network) ในโมเดลภาษาหลักให้เป็น Sparse MoE Layer โดยใช้ Top-2 Gating Router ในการจัดสรรให้โทเค็นวิ่งเข้าไปยังผู้เชี่ยวชาญ (Experts) 4 ด้านแยกขาดจากกันอย่างอิสระเพื่อประสิทธิภาพสูงสุด:

ผู้เชี่ยวชาญ (Expert)	หน้าที่หลัก	แหล่งข้อมูลสำคัญ
Expert 1: Audio	ตีความเสียง คลื่นเสียง ไวยากรณ์คำพูดไทย	Google FLEURS (th_th)
Expert 2: Vision/OCR	อ่านลายมือ สแกนภาพ โครงสร้างเอกสารไทย	LLaVA-Instruct-150K / ColPali
Expert 3: Text/Lang	ความเข้าใจหลักภาษา การสนทนาทั่วไป	FineTome-100k
Expert 4: Agent	การเรียกใช้เครื่องมือ คิววิเคราะห์แบบ ReAct	xverse/agent-sft-data (ไทย)

3. ขั้นตอนการพัฒนาและฝึกสอน (Training Phases)

กระบวนการฝึกสอนถูกจำแนกออกเป็น 4 เฟสหลัก เพื่อไม่ให้รับทวนความสามารถทางภาษาดั้งเดิมของโมเดล:

เฟส (Phase)	ขอบเขตการเทรน / ปลดล็อกน้ำหนัก	เป้าหมายหลัก
Phase 1: Audio Alignment	AudioProjector MLP เท่านั้น ส่วนอื่นแช่แข็งทั้งหมด	แมปสัญญาณเสียงไทยเข้าหามิตคำศัพท์หลัก ของ LLM (1024-dim)
Phase 2: Omni SFT	LLM Backbone + AudioProjector Audio/Vision Encoder แช่แข็ง	ฝึกฝนความเข้าใจแบบรวมประสาทสัมผัส โดยใช้ข้อมูลสัดส่วนผสม (Audio 35%, Text 35%, Vision 30%)
Phase 3: Distillation	หัวใจระบบประมวลผลของโมเดลลูก	ส่งต่อโครงสร้างและความเร็วไปยังหัวประมวล ผลเสียงพูดแบบ Streaming
Phase 4: Native Speech Out	AudioLMHead เท่านั้น (~5.2M พารามิเตอร์)	แปลง Hidden States เป็นรหัสคำพูดตอบกลับเสียงสดไทยโดยตรง

4. การเลือกชุดข้อมูลและการปรับแก้ปัญหาเสถียรภาพ

ในระหว่างการดำเนินโครงการ ทีมงานได้เผชิญหน้าและก้าวผ่านปัญหาทางเทคนิคสำคัญหลายประการ:

4.1 ปัญหาการฝังข้อมูลเสียงล้มเหลว (Training on Silence)

ปัญหา: ชุดข้อมูลเริ่มต้น (Typhoon Audio Preview) มีเฉพาะข้อมูลระดับ Metadata (Path ไฟล์บน SCB-10X) แต่ไม่มีไฟล์คลื่นเสียงดิบ ส่งผลให้ระบบใช้ตัวแปรเสียงศูนย์ (Zero-tensor) และเทรนโมเดลบนความเงียบ รวมถึงมี `NameError` ในระบบกู้ภัยข้อผิดพลาด

แนวทางแก้ไข: ย้ายระบบการจัดตำแหน่งเสียงมาใช้ชุดข้อมูล Google FLEURS (th th) ซึ่งมีเสียงสดฝังอยู่ในระบบ Parquet ของ Hugging Face พร้อมแก้ไขโครงสร้างการดึงตัวแปรเสียงเพื่อลดข้อผิดพลาดทั้งหมด

4.2 ปัญหา Dependency ของระบบเสียงและการหลีกเลี่ยง torchcodec

ปัญหา: ไลบรารี datasets ของ Hugging Face มีการนำเข้า torchcodec แบบ Lazy ซึ่งจะกระตุ้นข้อผิดพลาด `ImportError` ใน `DataLoader workers` เครื่อง Cloud ที่ขาดไลบรารีระบบ FFmpeg

แนวทางแก้ไข: สแกนคอลัมน์และแคสต์ประเภทคอลัมน์ที่เป็น Audio ทั้งหมดในขั้นตอนโหลดข้อมูลให้เป็น Plain Python Dictionary (มีโครงสร้าง bytes และ path) ถาวร จากนั้นทำการถอดรหัสเสียงดิบด้วยตนเองโดยใช้ไลบรารี `soundfile` และ `io.BytesIO` รวมถึงทำการจำลอง Downmix สเตอริโอเป็นโมโนเพื่อลดความคลาดเคลื่อนของขนาด

4.3 ปัญหาโครงสร้างคอลัมน์ของชุดข้อมูลเสียงสนทนา (KeyError: 'response')

ปัญหา: ชุดข้อมูล typhoon-ai/chatbot-arena-spoken-voices มีโครงสร้างคอลัมน์เสียงแตกต่างจากปกติ (เช่น `voice_a`, `voice_user`) และไม่มีคอลัมน์ `response` ส่งผลให้เกิด `KeyError` ในขั้นตอน SFT

แนวทางแก้ไข: เพิ่มระบบตรวจสอบประเภทข้อมูลใน `_getitem_` ของ `DataLoader` เมื่อพบรูปแบบข้อมูลบทสนทนาจะสกัดเสียงพูดและทรานสคริปต์ของคำตอบผู้ช่วย (Assistant Response) จาก `conversation_a/voice_a` ออกมาใช้งานแบบพลวัตโดยอัตโนมัติ

4.4 ข้อผิดพลาดชนิดข้อมูลของเลเยอร์ MoE (BFloat16 vs Float32)

ปัญหา: เกิดข้อผิดพลาด `RuntimeError: index_add(): self (BFloat16) and source (Float) must have the same scalar type` ในโมดูล `moe.py` เนื่องจากผลลัพธ์ของฟังก์ชัน `Softmax` จากเกตถูกแปลงเป็น `Float32` เพื่อเสถียรภาพตัวเลข ในขณะที่ค่า Hidden States ของแบบจำลองอยู่ในโหมด `BFloat16`

แนวทางแก้ไข: เพิ่มโค้ดแคสต์ชนิดข้อมูล (`expert_outputs * expert_weights`).to(`final_output.dtype`) ก่อนนำเข้าการประมวลผลคำสั่ง `index_add_` เพื่อรองรับความเข้ากันได้แบบ 100%

4.5 การเตรียมความพร้อมการย้ายระบบประมวลผล (Session Switching Ready)

พัฒนาและเขียนสคริปต์แบบรวมศูนย์ scripts/start_phase2.py เพื่อรองรับการทำงานย้ายข้ามเซสชัน GPU โดยอัตโนมัติ ซึ่งจะครอบคลุมขั้นตอนล็อกอิน ดาวน์โหลดน้าหนัก ทดสอบคุณภาพเสียง (Inference Check) และสตาร์ท SFT ทันที

5. แผนการพัฒนาในอนาคต (Roadmap)

เฟสปัจจุบัน	ตรวจสอบเสถียรภาพโมเดลบน Phase 1 พร้อมอัปโหลดน้ำหนักตัวจัดวางเสียงไทย
มิถุนายน 2026	รัน Phase 2 SFT บน GPU Session ใหม่ด้วยความยาวข้อมูลรวมสูงสุด 1024 โทเค็น
กรกฎาคม 2026	แปลงโครงสร้าง Qwen3.5 เป็น 4-Expert MoE โดยการแข่งความรู้ปกติและปรับแต่ง Router
สิงหาคม 2026	ฝึกสอนขั้นสุดท้ายสำหรับหัวส่งออกเสียงภาษาไทยสด (Native Speech Streaming out)